

🔑 Objectifs de la feuille

- Développement de fonctionnalités en équipe
- Pull Request / Code Review / Merge
- Nettoyage / Formatage / Refactoring
- Release / Tags / Versions

Présentation

Ce projet consiste à travailler avec Git dans une équipe de développement et à réaliser le re-factoring pour maintenir la qualité du code au cours de la vie du projet/produit. Il n'est pas forcément nécessaire de connaître toutes les commandes de GIT mais il faut surtout maîtriser les concepts et savoir à quel moment il faut exécuter une action précise. Pour le refactoring, vous devez améliorer la qualité du code existante, en utilisant les outils vus précédemment sur le projet n°1.

Contexte du projet

Nous allons vous fournir le code de base d'un jeu AirPlane Wars. Ce code de base n'est pas complet et il y a de nombreux problèmes de la qualité. Vous devez compléter le code et améliorer la qualité du code dans les étapes suivantes.

Dans ce projet, vous travailler dans une équipe de 3 personnes : 2 développeurs (un binôme d'étudiants) et l'enseignant. Il vous faut identifier au début qui est le développeur 1 et le développeur 2.

Ce TP noté correspond à 35% de la note finale. La note est basé sur tous les commits que vous allez réaliser. Alors pour avoir une bonne note il faut bien attention sur les points suivants :

- Le message d'un commit doit être très clair, précis mais il ne faut pas trop long.
- Les commits pendant les séances de TP sont mieux notés. Les commits en dehors de ces heures ont un malus de 20%.
- Avoir les docstring dans tous les fichiers, les classes et les fonctions importantes.
- Avoir les commentaires au cas où vous ne pouvez pas expliquer le but par le code soi-même.
- Les outils Linter (pylint) et Formater (yapf) ne trouve pas de problèmes dans votre code.

Le projet est évalué directement sur le dépôt Git où vous ferez des commits, pendant les séances de TP. Nous vous offrons trois jours de retard gratuits : les commits passés ce délai de 72h après la dernière séance du projet ne sont plus pris en compte.

Le partage volontaire de votre travail constitue une violation du règlement et le risque est la note 0.

Récupérer le code du projet

Le code se trouve dans CELENE dans l'archive [R3.04-proj02-airplan.zip](#). Elle contient 1 fichier principal et des répertoires ressources (sons, images et police).

Vous allez travailler en binôme. Vous devez créer un dépôt GitHub qui aura pour nom celui de votre binôme « NOM1-NOM2 ». Invitez ensuite votre enseignant sur le dépôt.

Début du développement

Il est recommandé d'utiliser Visual Studio Code pour éditer le code. Installez également l'extension SonarLint pour VSC.

- Le développeur 1 va mettre à jour le fichier readme.md pour ajouter vos informations, et vos numéros de développeur, directement dans la branche main et commiter les modifications au dépôt distant.
Il faut utiliser le langage Markdown (<https://daringfireball.net/projects/markdown/syntax>)
- Le développeur 2 va créer la branche develop et commiter au dépôt distant

A partir de maintenant, quand on dit commiter des modifications on comprend que ces commits sont aussi envoyés au dépôt distant. Vous utiliserez la méthode Git-Flow présentée dans les slides du cours et/ou dans le tutoriel <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>. L'objectif du projet est d'assurer la qualité du code et pratiquer le Git-Flow dans une équipe.

Explication de la base du code

La qualité du code fourni n'est pas optimale. Une des tâches à faire dans ce projet est d'améliorer la qualité du code et de réaliser le re-factoring quand nécessaire.

Étape 1 : La première étape consiste à afficher l'arrière-plan (code existant dans le code de base).

L'image de fond est chargée dans un premier temps avant la boucle principale d'exécution du jeu.

```
# Load the background map
background = pygame.image.load('resources/image/background.png')
```

Dans un seconde temps, cette image de fond est inséré à l'écran du jeu dans la boucle principale d'exécution du jeu.

```
while True:
    # Draw the background
    screen.fill(0)
    screen.blit(background, (0, 0))
```

Étape 2 : La seconde étape consiste à montrer l'avion du joueur (code existant dans le code de base).

L'image de l'avion est chargée dans un premier temps avant la boucle principale d'exécution du jeu.

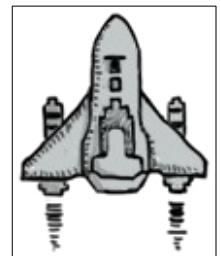
```
# Load the picture of the plane
plane_img = pygame.image.load('resources/image/shoot.png')
```

Le joueur est représentée par l'image de son avion, qui n'est autre qu'une sous-surface rectangulaire, une partie de l'image de départ **shoot.png**, basée sur des coordonnées (left, top, right, bottom).

```
# Select the position of the plane in the big picture, generate subsurface, and then initialize the position of the plane.
player_rect = pygame.Rect(0, 99, 102, 126)
player = plane_img.subsurface(player_rect)
```

Aussi, la position du joueur dans l'écran du jeu est initialisée via des coordonnées (x,y)

```
player_pos = [200, 600]
```



Dans un seconde temps, le joueur est inséré à l'écran du jeu dans la boucle principale d'exécution du jeu.

```
# Draw an airplane
screen.blit(player, player_pos)
```

Dans les étapes suivantes, il ne faut pas juste copier le code mais il faut savoir où le mettre et améliorer le code.

Étape 3 : Déplacer l'avion du joueur

- Le développeur 1 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 2.
- Le développeur 2 va travailler sur cette issue, autrement dit :
 - récupérer le contenu de develop : `git pull -all & git checkout develop`
 - créer une branche move_plane : `git checkout -b move_plane`
 - Implémenter la fonctionnalité avec les éléments de code suivant :

```
clock = pygame.time.Clock()
# Contrôlez la fréquence d'images maximale du jeu
clock.tick(45)
```

```
# Monitor keyboard events
key_pressed = pygame.key.get_pressed()
if key_pressed[K_UP]:
    player_pos[1] -= 3
if key_pressed[K_DOWN]:
    player_pos[1] += 3
if key_pressed[K_LEFT]:
    player_pos[0] -= 3
if key_pressed[K_RIGHT]:
    player_pos[0] += 3
```

- commiter les modifications
- créer une Pull Request (PR) de cette branche `move_plane` à la branche `develop`
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 1 comme le Reviewer
- le développeur 1 revoit le code et fait un retour au développeur 2 si nécessaire. Dans ce cas, le développeur 2 doit corriger les problèmes en se basant sur le retour du développeur 1. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 1 va merger la PR si tout est ok. Ne pas supprimer la branche `move_plane` !!

Étape 4 : Dessiner les avions ennemis et les faire avancer

- Le développeur 2 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 1.
- Le développeur 1 va travailler sur cette issue, autrement dit :
 - recupérer le contenu de develop : **git checkout develop & git pull origin develop**
 - créer une branche enemy_planes : **git checkout -b enemy_planes**
 - Implémenter la fonctionnalité avec les éléments de code suivant :

```
# Enemy class
class Enemy(pygame.sprite.Sprite):
    def __init__(self, enemy_img, init_pos):
        pygame.sprite.Sprite.__init__(self)
        self.image = enemy_img
        self.rect = self.image.get_rect()
        self.rect.topleft = init_pos
        self.speed = 2
        self.down_index = 0

    def move(self):
        self.rect.top += self.speed
```

```
# Définir les paramètres liés à la surface utilisés par l'objet avion ennemi
enemy1_rect = pygame.Rect(534, 612, 57, 43)
enemy1_img = plane_img.subsurface(enemy1_rect)
enemies1 = pygame.sprite.Group()
enemy_frequency = 0
```



```
# Déplacez l'avion ennemi, s'il dépasse la plage de la fenêtre, supprimez-le
for enemy in enemies1:
    enemy.move()
    if enemy.rect.top > SCREEN_HEIGHT:
        enemies1.remove(enemy)
enemies1.draw(screen)
```

```
# Faire apparaître des avions ennemis :
if enemy_frequency % 50 == 0:
    enemy1_pos = [random.randint(0, SCREEN_WIDTH - enemy1_rect.width), 0]
    enemy1 = Enemy(enemy1_img, enemy1_pos)
    enemies1.add(enemy1)
    enemy_frequency += 1
if enemy_frequency >= 100:
    enemy_frequency = 0
```

- commiter les modifications
 - résoudre les conflits s'ils existent
 - créer une Pull Request (PR) de cette branche **enemy_planes** à la branche **develop**
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 2 comme le Reviewer
- le développeur 2 revoit le code et fait un retour au développeur 1 si nécessaire. Dans ce cas, le développeur 1 doit corriger les problèmes en se basant sur le retour du développeur 2. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 2 va merger la PR si tout est ok. Ne pas supprimer la branche **enemy_planes** !!

Étape 5 Re-factoring : Créer une classe pour l'avion du joueur

- Le développeur 1 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 2.
- Le développeur 2 va travailler sur cette issue, autrement dit :
 - récupérer le contenu de develop : `git pull -all & git checkout develop`
 - créer une branche player : `git checkout -b player`
 - créer une classe Player qui représente l'avion du joueur : les caractéristiques de l'avion sont les images, la position de l'avion et la vitesse de l'avion. L'avion peut se déplacer en 4 directions donc on va ajouter 4 méthodes à la classe Player : `move_up()`, `move_down()`, `move_left()`, `move_right()`.

```
# Player
class Player (pygame.sprite.Sprite):
    def __init__(self, plane_img, player_rect, init_pos):
        pygame.sprite.Sprite.__init__(self)
        self.image = [] # List of pictures of player object wizard

        for i in range(len(player_rect)):
            self.image.append(plane_img.subsurface(player_rect[i]).convert_alpha())

        self.rect = player_rect[0] # Initialize the rectangle where the picture is located
        self.rect.topleft = init_pos # Initialize the upper left corner coordinates of the rectangle
        self.speed = 8 # Initialize the player speed, here is a definite value.
        self.img_index = 0 # Player Wizard Image Index

    def moveUp (self):
        if self.rect.top <= 0:
            self.rect.top = 0
        else:
            self.rect.top -= self.speed

    def moveDown(self):
        if self.rect.top >= SCREEN_HEIGHT - self.rect.height:
            self.rect.top = SCREEN_HEIGHT - self.rect.height
        else:
            self.rect.top += self.speed

    def moveLeft (self):
        if self.rect.left <= 0:
            self.rect.left = 0
        else:
            self.rect.left -= self.speed

    def moveRight (self):
        if self.rect.left >= SCREEN_WIDTH - self.rect.width:
            self.rect.left = SCREEN_WIDTH - self.rect.width
        else:
            self.rect.left += self.speed
```

- définir les images de l'avion du joueur : ces images sont présentées dans le fichier `shoot.png`. Les coordonnées de ces images sont présentées dans le fichier `shoot.pack`. En, plus de l'image de départ, on va ajouter 5 images de l'avion explosé

```
# Définir les paramètres liés au joueur
player_rect = []
player_rect.append(pygame.Rect(0, 99, 102, 126)) # Zone d'image du sprite du joueur
player_rect.append(pygame.Rect(165, 360, 102, 126))
player_rect.append(pygame.Rect(165, 234, 102, 126)) # Zone d'image du sprite d'explosion du joueur
player_rect.append(pygame.Rect(330, 624, 102, 126))
player_rect.append(pygame.Rect(330, 498, 102, 126))
player_rect.append(pygame.Rect(432, 624, 102, 126))
player_pos = [200, 600]
player = Player(plane_img, player_rect, player_pos)
```



- copier tous les éléments de code ci-dessous et les mettre au bon endroit (il n'est pas difficile à trouver le bon endroit). Attention, c'est un cours de qualité de développement. Il faut alors bien structurer votre code.

```
# Draw an airplane
# screen.blit (player, player_pos)
screen.blit(player.image[player.img_index], player.rect)
```

```
key_pressed = pygame.key.get_pressed()
if key_pressed[K_w] or key_pressed[K_UP]:
    player.moveUp()
if key_pressed[K_s] or key_pressed[K_DOWN]:
    player.moveDown()
if key_pressed[K_a] or key_pressed[K_LEFT]:
    player.moveLeft()
if key_pressed[K_d] or key_pressed[K_RIGHT]:
    player.moveRight()
```

- commiter les modifications et résoudre les conflits s'ils existent
- créer une Pull Request (PR) de cette branche `player` à la branche `develop`
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 1 comme le Reviewer
- le développeur 1 revoit le code et fait un retour au développeur 2 si nécessaire. Dans ce cas, le développeur 2 doit corriger les problèmes en se basant sur le retour du développeur 1. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 1 va merger la PR si tout est ok. Ne pas supprimer la branche `player` !!

Étape 6 : Dessiner les munitions envoyées par notre avion

- Le développeur 2 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 1.
- Le développeur 1 va travailler sur cette issue, autrement dit :
 - recupérer le contenu de develop : `git checkout develop & git pull origin develop`
 - créer une branche bullets : `git checkout -b bullets`
 - créer une classe Bullet

```
# Bullet
class Bullet (pygame.sprite.Sprite):
    def __init__(self, bullet_img, init_pos):
        pygame.sprite.Sprite.__init__(self)
        self.image = bullet_img
        self.rect = self.image.get_rect()
        self.rect.midbottom = init_pos
        self.speed = 10

    def move (self):
        self.rect.top -= self.speed
```

- modifier la classe Player pour ajouter l'attribut bullets et la méthode shoot()

```
# Player
self.bullets = pygame.sprite.Group() # Collection of bullets fired by the player's aircraft

def shoot (self, bullet_img):
    bullet = Bullet (bullet_img, self.rect.midtop)
    self.bullets.add (bullet)
```

- charger le son et l'image de la munition 

```
bullet_sound = pygame.mixer.Sound('resources/sound/bullet.wav')
bullet_sound.set_volume(0.3)
# Définir les paramètres liés à la surface utilisés par l'objet puce (bullets)
bullet_rect = pygame.Rect(1004, 987, 9, 21)
bullet_img = plane_img.subsurface(bullet_rect)
shoot_frequency = 0
```

- modifier la boucle principale pour contrôler la fréquence des tirs et faire avancer les puces.

```
# Contrôler la fréquence des tirs de balles et des balles de feu
if shoot_frequency % 15 == 0:
    bullet_sound.play()
    player.shoot(bullet_img)
shoot_frequency += 1
if shoot_frequency >= 15:
    shoot_frequency = 0
```

```
# Déplacer la puce, la supprimer si elle dépasse le cadre de la fenêtre
for bullet in player.bullets:
    bullet.move()
    if bullet.rect.bottom < 0:
        player.bullets.remove(bullet)
```

- dessiner les puces

```
# Draw the bullets
player.bullets.draw(screen)
```


- commiter les modifications
- résoudre les conflits s'ils existent
- créer une Pull Request (PR) de cette branche **bullets** à la branche **develop**
 - mettre une description expliquant ce qui a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 2 comme le Reviewer
- le développeur 2 revoit le code et fait un retour au développeur 1 si nécessaire. Dans ce cas, le développeur 1 doit corriger les problèmes en se basant sur le retour du développeur 2. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 2 va merger la PR si tout est ok. Ne pas supprimer la branche **bullets** !!

Étape 7 : Création de la branche de version

Les étapes 3,4,5 et 6 prévues pour la version 1.0 du logiciel se trouvent toutes dans la branche **develop**. Il est donc temps de créer la branche de version qui permettra de tester la prochaine version et d'y intégrer des correctifs avant sa publication. Le développeur 1 réalise cette tâche :

- **git checkout develop**
- **git pull**
- **git checkout -b release-v1.0**
- **git commit --allow-empty -m "Début branche de version 1.0"**
- **git push -u origin release-v1.0**

Une fois la branche créée, le développeur 1 ajoute un commentaire prévenant le développeur 2 que la branche **release-v1.0** a été créée et qu'elle est prête à être testée. Le développeur 2 teste la branche et peut y apporter des correctifs autorisés par le développeur 1. Si les tests sont jugés concluants, c-à-d, que le développeur 2 estime qu'il a testé toutes les fonctionnalités, la branche **release-v1.0** est déclarée suffisamment stable pour devenir la nouvelle version stable du logiciel.

Le développeur 1 se charge alors de créer le commit de la version 1.0

- **git checkout master**
- **git merge --no-ff release-v1.0 -m "Nouvelle version 1.0"**
- **git push**
- **git tag v1.0**
- **git push --tags**

Et enfin, le développeur 2 met également à jour la branche **develop** pour qu'elle reçoive les correctifs contenus dans la branche de version :

- **git checkout develop**
- **git merge --no-ff release-v1.0 -m "Correctifs v1.0"**
- **git push**
- **git branch -d release-v1.0**
- **git push origin :release-v1.0**

Étape 8 : Exploder les avions ennemis si touchés par une munition

- Le développeur 1 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 2.
- Le développeur 2 va travailler sur cette issue, autrement dit :
 - recupérer le contenu de develop : `git pull -all & git checkout develop`
 - créer une branche enemy_down : `git checkout -b enemy_down`
 - modifier le constructeur de la classe `Enemy` pour ajouter le paramètre représentant les images de l'explosion des avions ennemis `enemy_down_imgs`

```
# Enemy class
class Enemy(pygame.sprite.Sprite):
    def __init__(self, enemy_img,
                 enemy_down_imgs,
                 init_pos):
        pygame.sprite.Sprite.__init__(self)
        self.image = enemy_img
        self.rect = self.image.get_rect()
        self.rect.topleft = init_pos
        self.down_imgs = enemy_down_imgs
        self.speed = 2
        self.down_index = 0
```

- charger le son et les images des ennemis explosées

```
enemy1_down_sound = pygame.mixer.Sound('resources/sound/enemy1_down.wav')
enemy1_down_sound.set_volume(0.3)
enemy1_down_imgs = []
enemy1_down_imgs.append(plane_img.subsurface(pygame.Rect(267, 347, 57, 43)))
enemy1_down_imgs.append(plane_img.subsurface(pygame.Rect(873, 697, 57, 43)))
enemy1_down_imgs.append(plane_img.subsurface(pygame.Rect(267, 296, 57, 43)))
enemy1_down_imgs.append(plane_img.subsurface(pygame.Rect(930, 697, 57, 43)))
# Stockez des avions détruits pour le rendu d'animations de sprites d'épaves
enemies_down = pygame.sprite.Group()
```



- modifier le code à l'endroit de l'instanciation des ennemis

```
enemy1 = Enemy(enemy1_img,
               enemy1_down_imgs, #Ajouter ceci car on a modifié la classe Enemy
               enemy1_pos)
```

- supprimer l'ennemi en cas de collision et ajouter `enemy_down` pour le dessiner

```
# Ajoutez l'objet avion ennemi touché au groupe d'avions ennemis détruits, utilisé pour rendre l'animation de destruction
enemies1_down = pygame.sprite.groupcollide(enemies1, player.bullets, 1, 1)
for enemy_down in enemies1_down:
    enemies_down.add(enemy_down)
```

- dessiner les ennemis explosés

```
# Dessinez l'animation de l'épave
for enemy_down in enemies_down:
    if enemy_down.down_index == 0:
        enemy1_down_sound.play()
    if enemy_down.down_index > 7:
        enemies_down.remove(enemy_down)
    # score += 1000
    continue
screen.blit(enemy_down.down_imgs[enemy_down.down_index // 2], enemy_down.rect)
enemy_down.down_index += 1
```

- commiter les modifications et résoudre les conflits s'ils existent
- créer une Pull Request (PR) de cette branche **enemy_down** à la branche **develop**
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 1 comme le Reviewer
- le développeur 1 revoit le code et fait un retour au développeur 2 si nécessaire. Dans ce cas, le développeur 2 doit corriger les problèmes en se basant sur le retour du développeur 1. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 1 va merger la PR si tout est ok. Ne pas supprimer la branche **enemy_down**

Etape 9 : Afficher Game Over si le joueur est touché

- Le développeur 2 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 1.
- Le développeur 1 va travailler sur cette issue, autrement dit :
 - récupérer le contenu de develop : **git checkout develop & git pull origin develop**
 - créer une branche game_over : **git checkout -b game_over**
 - ajouter un attribut **is_hit** dans la classe **Player** et le positionner à **False**.

```
self.is_hit = False # Is the player hit?
```

- déterminer si le joueur a été touché

```
# Déplacez l'avion ennemi, s'il dépasse la plage de la fenêtre, supprimez-le
for enemy in enemies1:
    enemy.move()
    # Déterminez si le joueur a été touché
    if pygame.sprite.collide_circle(enemy, player):
        enemies_down.add(enemy)
        enemies1.remove(enemy)
        player.is_hit = True
        game_over_sound.play()
    Break
```

- remplacer le code pour dessiner le joueur par ce code plus complexe

```
# dessiner l'avion du joueur
if not player.is_hit:
    screen.blit(player.image[player.img_index], player.rect)
    # Changer l'index de l'image pour animer l'avion
    player.img_index = shoot_frequency // 8
else:
    player.img_index = player_down_index // 8
    screen.blit(player.image[player.img_index], player.rect)
    player_down_index += 1
    if player_down_index > 47:
        running = False
```

- ajouter cette ligne `player_down_index = 16`
- charger la photo pour afficher quand le jeu est game over

```
game_over = pygame.image.load('resources/image/gameover.png')
game_over_sound = pygame.mixer.Sound('resources/sound/game_over.wav')
game_over_sound.set_volume(0.3)
```

- Ajouter ce code à la fin du programme pour afficher le Game Over

```
screen.blit(game_over, (0, 0))

while 1:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
    pygame.display.update()
```

- commiter les modifications
- résoudre les conflits s'ils existent
- créer une Pull Request (PR) de cette branche `game_over` à la branche `develop`
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 2 comme le Reviewer
- le développeur 2 revoit le code et fait un retour au développeur 1 si nécessaire. Dans ce cas, le développeur 1 doit corriger les problèmes en se basant sur le retour du développeur 2. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 2 va merger la PR si tout est ok. Ne pas supprimer la branche `game_over` !!

Etape 10 :Afficher le nombre de points gagnés

- Le développeur 1 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 2.
- Le développeur 2 va travailler sur cette issue, autrement dit :
 - récupérer le contenu de develop : `git pull -all & git checkout develop`
 - créer une branche scoring : `git checkout -b scoring`
 - ajouter une variable globale `score`

```
score = 0
```

- augmenter le score si un ennemi est explosé

```
# Dessinez l'animation de l'épave
for enemy_down in enemies_down:
    if enemy_down.down_index == 0:
        enemy1_down_sound.play()
    if enemy_down.down_index > 7:
        enemies_down.remove(enemy_down)
        score += 1000
        continue
    screen.blit(enemy_down.down_imgs[enemy_down.down_index // 2], enemy_down.rect)
    enemy_down.down_index += 1
```

- dessiner le score dans la boucle principale

```
# dessiner le score
score_font = pygame.font.Font(None, 36)
score_text = score_font.render(str(score), True, (128, 128, 128))
text_rect = score_text.get_rect()
text_rect.topleft = [10, 10]
screen.blit(score_text, text_rect)
```

- dessiner le score lorsque la partie est terminée

```
font = pygame.font.Font(None, 48)
text = font.render('Score: ' + str(score), True, (255, 0, 0))
text_rect = text.get_rect()
text_rect.centerx = screen.get_rect().centerx
text_rect.centery = screen.get_rect().centery + 24
screen.blit(game_over, (0, 0))
screen.blit(text, text_rect)
while 1:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```

- commiter les modifications et résoudre les conflits s'ils existent
- créer une Pull Request (PR) de cette branche **scoring** à la branche **develop**
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 1 comme le Reviewer
- le développeur 1 revoit le code et fait un retour au développeur 2 si nécessaire. Dans ce cas, le développeur 2 doit corriger les problèmes en se basant sur le retour du développeur 1. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 1 va merger la PR si tout est ok. Ne pas supprimer la branche **scoring**

Etape 11 : Ajouter une fonctionnalité supplémentaire

Voici les fonctionnalités complémentaires que vous pouvez ajouter dans le jeu.

- a. Faire bouger le background comme l'avion avance bien dans l'espace
 - b. Changer le niveau du jeu (changer la vitesse des puces, changer la vitesse des ennemies, ajouter les bombes délivrées par des ennemies)
 - c. Changer la puissance de l'avion ou changer l'avion (voir des images d'avion dans ressources/image/shoot.png) si on arrive à 50 000 points
 - d. ajouter d'autres types d'armes pour l'avion
- Le développeur 2 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 1.
 - Le développeur 1 va travailler sur cette issue, autrement dit :
 - récupérer le contenu de develop : **git checkout develop & git pull origin develop**
 - créer une branche bonus : **git checkout -b bonus**
 - implémenter la fonctionnalité
 - commiter les modifications
 - résoudre les conflits s'ils existent
 - créer une Pull Request (PR) de cette branche **bonus** à la branche **develop**
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 2 comme le Reviewer
 - le développeur 2 revoit le code et fait un retour au développeur 1 si nécessaire. Dans ce cas, le développeur 1 doit corriger les problèmes en se basant sur le retour du développeur 2. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
 - le développeur 2 va merger la PR si tout est ok. Ne pas supprimer la branche **bonus** !!

Etape 12 : Formatage (YAPF) et Refactoring (SonarLint ou Pylint)

- Le développeur 1 va créer une « github issue » pour cette fonctionnalité et va l'assigner au développeur 2
- Le développeur 2 va travailler sur cette issue, autrement dit :
 - récupérer le contenu de develop : `git checkout develop & git pull origin develop`
 - créer une branche refactoring : `git checkout -b refactoring`
 - Résoudre les warnings du SonarLint, améliorer la note de Pylint, formater
 - commiter les modifications
 - résoudre les conflits s'ils existent
 - créer une Pull Request (PR) de cette branche **bonus** à la branche **develop**
 - mettre une description expliquant ce que a été fait, y compris l'amélioration de la qualité du code.
 - assigner le développeur 2 comme le Reviewer
- le développeur 1 revoit le code et fait un retour au développeur 1 si nécessaire. Dans ce cas, le développeur 2 doit corriger les problèmes en se basant sur le retour du développeur 2. **Pendant la revue de code, et les éventuelles corrections, communiquez exclusivement par GITHUB, ne questionnez pas et ne faites pas de commentaires directement (même si vous êtes côte à côte).**
- le développeur 1 va merger la PR si tout est ok. Ne pas supprimer la branche **refactoring** !!

Etape 13 : Monter en version 2.0

Les étapes 8,9,10,11 et 12 prévues pour la version 2.0 du logiciel se trouvent toutes dans la branche **develop**. Il est donc temps de créer la branche de version qui permettra de tester la prochaine version et d'y intégrer des correctifs avant sa publication. Le développeur 1 réalise cette tâche :

- `git checkout develop`
- `git pull`
- `git checkout -b release-v2.0`
- `git commit --allow-empty -m "Début branche de version 2.0"`
- `git push -u origin release-v2.0`

Une fois la branche créée, le développeur 1 ajoute un commentaire prévenant le développeur 2 que la branche **release-v2.0** a été créée et qu'elle est prête à être tester. Le développeur 2 teste la branche et peut y apporter des correctifs autorisés par le développeur 1. Si les tests sont jugés concluants, c-a-d, que le développeur 2 estime qu'il a testé toutes les fonctionnalités, la branche **release-v2.0** est déclarée suffisamment stable pour devenir la nouvelle version stable du logiciel.

Le développeur 1 se charge alors de créer le commit de la version 2.0

- `git checkout master`
- `git merge --no-ff release-v2.0 -m "Nouvelle version 2.0"`
- `git push`
- `git tag v2.0`
- `git push --tags`

Et enfin, le développeur 2 met également à jour la branche `develop` pour qu'elle reçoive les correctifs contenus dans la branche de version :

- `git checkout develop`
- `git merge --no-ff release-v2.0 -m "Correctifs v2.0"`
- `git push`
- `git branch -d release-v2.0`
- `git push origin :release-v2.0`